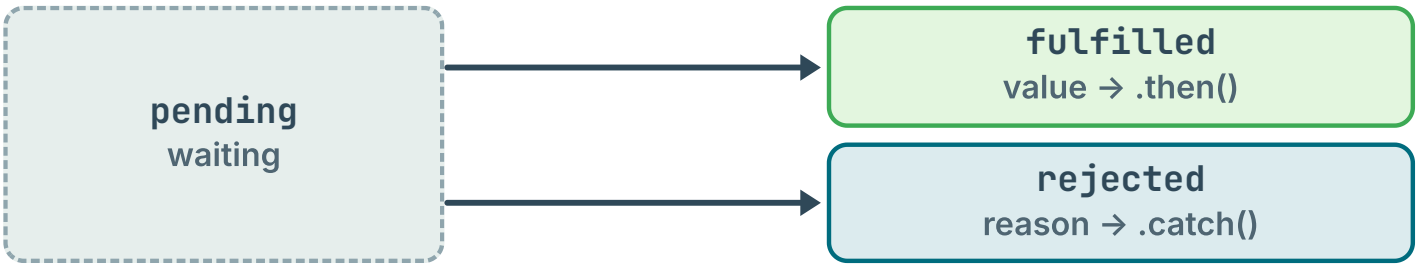
# Promises

Three states, one chain, and values flow down

## A promise settles once




The executor runs now, **then** callbacks later as microtasks. A **then** can return a promise: the chain waits for it.

A rejection skips every **then** until the next **catch**. **.finally()** runs after either one.

## 1 Values flow down

```
Promise.resolve(1)
  .then(n => n + 1)
  .then(n => n * 10)
  .then(console.log);   // 20
// each then gets the last value
```

## 2 catch recovers

```
Promise.reject(new Error("x"))
  .then(() => "skipped")
  .catch(() => "recovered")
  .then(console.log);
// recovered: chain is fulfilled
```

## TRAP What does this log?

```
new Promise((res, rej) => {
  res(1); rej(2); res(3);
}).then(console.log);
```

**1**

A promise settles only once. Later calls are ignored.

**KEY TAKEAWAY**

- **then** — Returns a new promise
- **catch** — Handles a rejection
- **finally** — Runs either way

## ? Follow-up questions to expect

- **What if a `then` forgets to return?** → The next then gets undefined.
- **`then(ok, fail)` vs `.catch(fail)`?** → fail misses errors thrown in ok.
- **What if `finally()` throws?** → That error replaces the result.
- **What if a rejection is never handled?** → Node 15+ exits with code 1.